

Parkino Mobile App

Documentation Complete du Projet

Date: 29 Mai 2026

Parkino Mobile App - Documentation Complète du Projet

Date: Mai 29, 2026
Projet: Parkino - Application de Gestion de Parking Intelligent
Plateforme: Flutter (Multi-plateforme: Android, iOS, Web, Windows, macOS, Linux)

■ Table des Matières

1. Structure Générale du Projet(#structure-générale) 2. Dossiers Principaux(#dossiers-principaux) 3. Fichiers du Répertoire Racine(#fichiers-racine) 4. Dossier /lib - Code Source(#dossier-lib) 5. Architecture et Patterns(#architecture)

Structure Générale du Projet

parkino-mobile-app/ ■■■ android/ # Configuration Android native ■■■ ios/ # Configuration iOS native ■■■ web/ # Configuration Web ■■■ windows/ # Configuration Windows ■■■ macos/ # Configuration macOS ■■■ linux/ # Configuration Linux ■■■ lib/ # Code source Flutter principal ■■■ test/ # Tests unitaires ■■■ assets/ # Ressources (images, fonts, etc.) ■■■ build/ # Artifacts compilés ■■■ pubspec.yaml # Dépendances du projet ■■■ Documentation files # Fichiers de documentation

Dossiers Principaux

■ Dossier /android

Rôle: Configuration et code natif spécifique à Android

| Fichier/Dossier | Description | |---|---| | build.gradle.kts | Configuration Gradle pour le build Android | | gradle.properties | Propriétés globales du build Gradle | | gradlew / gradlew.bat | Scripts d'exécution Gradle | | local.properties | Configuration locale Android SDK | | settings.gradle.kts | Configuration des modules Gradle | | app/ | Code source de l'application Android | | gradle/ | Configuration Gradle avancée |

■ Dossier /ios

Rôle: Configuration et code natif spécifique à iOS

| Fichier/Dossier | Description | |---|---| | Flutter/ | Framework Flutter pour iOS | | Runner/ | Projet Xcode principal | | Runner.xcodeproj/ | Fichier projet Xcode | | Runner.xcworkspace/ | Workspace Xcode | | RunnerTests/ | Tests iOS unitaires |

■ Dossier /web

Rôle: Configuration et ressources pour version web

| Fichier | Description | |---|---| | index.html | Page HTML principale | | manifest.json | Manifeste PWA | | icons/ | Icônes de l'application web |

■ Dossiers /windows, /macos, /linux

Rôle: Configuration des versions desktop

| Contenu | Description | |---|---| | CMakeLists.txt | Configuration de compilation C++ | | flutter/ | Configuration Flutter | | runner/ | Code natif de l'application |

■ Dossier /assets

Rôle: Ressources statiques (images, fonts, etc.)

| Dossier | Description | |---|---| | images/ | Images PNG, SVG, logos |

■ Dossier /build

Rôle: Artifacts compilés (générés automatiquement)

| Dossier | Description | |---|---| | app/ | Build compilé | | flutter_assets/ | Assets Flutter compilés | | reports/ | Rapports de test et analyse | | Autres | Dépendances compilées |

■ Dossier /test

Rôle: Tests automatisés

| Fichier | Description | |---|---| | widget_test.dart | Tests des widgets Flutter |

Fichiers du Répertoire Racine

| Fichier | Description | |---|---| | pubspec.yaml | Configuration des dépendances Flutter | | analysis_options.yaml | Options d'analyse de code Dart | | README.md | Documentation générale du projet | | test_app.iml | Fichier IntelliJ IDEA | | test_app_android.iml | Fichier IntelliJ IDEA pour Android |

Fichiers de Documentation

| Fichier | Description | |---|---| | ARCHITECTURE_DIAGRAM.md | Diagramme d'architecture | | COMPLETE_FLOW_EXAMPLE.md | Exemple complet de flux utilisateur | | FIRESTORE_INTEGRATION_GUIDE.md | Guide d'intégration Firestore | | PARKING_SYSTEM_README.md | Documentation du système de parking | | IMPLEMENTATION_GUIDE.md | Guide d'implémentation | | TESTING_GUIDE.md | Guide de test | | QUICK_START_COUNTDOWN.md | Quickstart pour le système de compte à rebours | | Et autres fichiers README... | Documentation spécifique aux fonctionnalités |

Configuration de Compilation

| Fichier | Description | |---|---| | local.properties | Propriétés locales (SDK, clés API, etc.) |

Dossier /lib - Code Source Principal

Le dossier /lib contient tout le code source Dart de l'application.

■ Fichiers Principaux

| Fichier | Description | |---|---| | main.dart | Point d'entrée de l'application - Configuration initiale, initialisation Firebase, setup des providers | | firebase_options.dart | Configuration Firebase (API keys, project ID, etc.) |

■ Sous-Dossiers et Leur Rôle

1. /lib/localization ■ Rôle: Gestion de la localisation (multi-langues)

| Fichier | Description | |---|---| | app_localizations.dart | Dictionnaire principal avec toutes les traductions (EN, FR, AR) |

Langues supportées: - English (en) - Français (fr) - ■■■■■■■■ (ar)

2. /lib/models ■ Rôle: Définition des modèles de données

| Fichier | Description | |---|---| | parking_slot.dart | Modèle pour une place de parking | | reservation.dart | Modèle pour une réservation complète | | payment.dart | Modèle pour un paiement | | slot_reservation.dart | Modèle pour réservation courte (5 min) | | index.dart | Fichier d'export de tous les modèles |

Structures principales: - ParkingSlot - Une place (disponible, occupée, réservée) - Reservation - Réservation de longue durée - SlotReservation - Réservation courte (5 minutes) - Payment - Informations de paiement

3. /lib/services ■■ Rôle: Logique métier et accès aux données

| Fichier | Description | |---|---| | parking_repository.dart | Accès Firestore pour les données de parking | | parking_models.dart | Modèles de données Firestore pour parking | | slot_reservation_service.dart | Service pour gérer réservations courtes (5 min) | | reservation_service.dart | Service pour gérer réservations longues | | reservation_timer_service.dart | Service pour gérer les timers de réservation | | payment_service.dart | Service de paiement (simulation) | | payment_database_service.dart | Gestion base de données paiements | | password_validation_service.dart | Service de validation de mots de passe | | password_reset_link_service.dart | Service pour réinitialisation mot de passe | | index.dart | Fichier d'export des services |

Services clés: - Parking Repository - Gère l'accès aux étages (Floor 1, Floor 2) et places - Slot Reservation - Réservations courtes avec codes 4 chiffres - Payment - Traitement des paiements (test: 4242... accepté) - Auth & Password - Authentification et validation

4. /lib/providers ■ Rôle: Gestion d'état avec Provider pattern

| Fichier | Description | |---|---| | firebase_auth_provider.dart | Authentification Firebase (Sign In, Sign Up) | | language_provider.dart | Gestion de la langue actuelle | | parking_provider.dart | État global des données de parking | | slot_reservation_provider.dart | État des réservations courtes | | reservation_notification_provider.dart | Notifications et timers de réservation | | reservation_provider.dart | État des réservations longues | | index.dart | Fichier d'export des providers |

Patterns: - ChangeNotifier - Pour la réactivité - Multi-provider dans main.dart - Listeners pour synchronisation d'état

5. /lib/screens ■ Rôle: Interface utilisateur - Écrans de l'application

Authentification (/screens/auth) | Fichier | Description | |---|---| | sign_in_screen.dart | Écran de connexion avec validation | | sign_up_screen.dart | Écran d'inscription avec validation mot de passe | |

password_reset_screen.dart | Écran de réinitialisation mot de passe |

Accueil (/screens/home) | Fichier | Description | |---|---| | home_screen.dart | Écran principal - Statut parking, compteur réservation |

Parking (/screens/map) | Fichier | Description | |---|---| | parking_map_screen.dart | Vue carte du parking par étage |

Réservation (/screens) | Fichier | Description | |---|---| | parking_reservation_hub.dart | Hub de gestion des réservations | | payment_screen.dart | Écran de paiement | | confirmation_screen.dart | Écran de confirmation avec QR code | | reservation_management_screen.dart | Gestion des réservations utilisateur |

Profil (/screens/profile) | Fichier | Description | |---|---| | profile_screen.dart | Export vers profile_screen_modern.dart | | profile_screen_modern.dart | Écran profil - Données utilisateur, image | | edit_profile_screen.dart | Édition du profil utilisateur |

Notifications (/screens/notifications) | Fichier | Description | |---|---| | notifications_screen.dart | Écran des notifications |

Index | Fichier | Description | |---|---| | index.dart | Export de tous les écrans |

6. /lib/widgets ■ Rôle: Composants réutilisables

| Fichier | Description | |---|---| | modern_widgets.dart | Widgets modernes (GlassCard, ModernButton, ModernTextField) | | language_button.dart | Bouton de changement de langue | | language_settings_sheet.dart | Bottom sheet de sélection langue | | password_validation_widgets.dart | Widgets de validation mot de passe | | reservation_countdown_widget.dart | Compteur décroissant pour réservations | | unique_reservation_guard_widget.dart | Avertissement réservation unique |

Widgets principaux: - GlassCard - Effet glassmorphism - ModernButton - Bouton animé - ModernTextField - Champ texte moderne - ReservationCountdownTimer - Affichage du compteur

7. /lib/navigation ■ Rôle: Navigation et routing

| Fichier | Description | |---|---| | main_navigation.dart | Navigation principale (BottomNavigationBar) |

Routes principales: - Home - Map - Stats - Notifications - Profile

8. /lib/theme ■ Rôle: Thème et styling global

| Fichier | Description | |---|---| | parkino_theme.dart | Thème complet - Couleurs, typos, ombres |

Couleurs principales: - Bleu foncé primaire: #0B2A4A - Jaune doré: #FFC107 - Blanc: #F4F7FA - Gris: #E9ECEF

9. /lib/Utils ■ Rôle: Utilitaires et helpers

| Fichier | Description | |---|---| | cleanup_firestore.dart | Nettoyage des collections Firestore orphelines |

Architecture et Patterns

■ Patterns Utilisés

1. Provider Pattern ChangeNotifier Providers: ■■■ FirebaseAuthProvider → Authentification ■■■ LanguageProvider → Localization ■■■ ParkingProvider → Données parking ■■■ SlotReservationProvider → Réservations courtes ■■■ NotificationProvider → Notifications ■■■ ReservationProvider → Réservations longues

2. Service Layer - Séparation logique métier des écrans - Services Firestore pour accès données - Services spécialisés (Payment, Auth, etc.)

3. Model Layer - Modèles JSON sérialisables - Enums pour statuts - Méthodes utilitaires (formatting, calculs)

4. UI Layer - Écrans (StatefulWidget, StatelessWidget) - Widgets réutilisables - Animations et transitions

■ Flux de Données

Réservation Courte (5 minutes) User → Home/Map Screen ↓ SlotReservationProvider.reserveSlot() ↓ SlotReservationService.reserveSlot() Firestore Transaction ↓ Slot update + SlotReservation document created ↓ NotificationProvider shows countdown ↓ User sees QR code / enters spot

Réservation Longue Payment → ReservationService ↓ Create Reservation (pending_payment) ↓ Payment processing ↓ Update Reservation (confirmed) + QR code ↓ ConfirmationScreen

■ Firebase Integration

Firestore Structure parkings/ ■■■■ main_parking/ ■ ■■■■ floors/ ■ ■ ■■■■ etage_1/ ■ ■ ■ ■■■■ slots/ slot_1 → slot_8 ■ ■ ■■■■ etage_2/ ■ ■ ■■■■ slots/ slot_1 → slot_6 ■ ■ ■■■■ slot_id/ ■ ■ ■■■■ slot_reservations/ 5-min reservations ■■■■ users/ user profiles + images en base64 ■■■■ reservations/ long-term reservations

Collections - parkings - Données de parking - reservations - Réservations longues - users - Profils utilisateurs - slot_reservations - Réservations courtes (sous-collection)

■ État de l'Application

Statuts de Place - free - Libre (vert) - occupied - Occupée (rouge) - isReserved: true - Réservée (bleu)

Statuts de Réservation - pending_payment - En attente de paiement - confirmed - Confirmée (payée) - used - Utilisée - expired - Expirée

Statuts SlotReservation - active - Actif (5 min) - expired - Expiré - used - Utilisé

■ UI/UX

Design System - Glassmorphism - Cards avec effet verre - Gradients - Dégradés bleu → doré - Animations - Transitions fluides - Responsive - Adapté à tous les écrans

Composants Clés - ModernButton - CTA principal - GlassCard - Conteneurs - ModernTextField - Inputs - ReservationCountdownTimer - Urgence

Dépendances Principales

Firestore

- firebase_core - Initialiseur Firestore - firebase_auth - Authentification - cloud_firestore - Base de données temps réel - firebase_storage - Stockage d'images

UI/UX

- google_fonts - Typographies modernes - animations - Animations flutter - lottie - Animations Lottie - cached_network_image - Images cachées

Utilitaires

- provider - Gestion d'état - image_picker - Sélection images - intl - Localisation dates - uuid - Génération IDs uniques

Points Clés du Projet

■ Authentification Firebase - Sign In, Sign Up, Password Reset ■ Réservations Courtes - 5 minutes avec code QR ■ Réservations Longues - Durées personnalisées ■ Système de Paiement - Simulation avec cartes test ■ Notifications - Compteur décroissant en temps réel ■ Multi-langue - EN, FR, AR ■ Profil Utilisateur - Avatar en base64, édition ■ Vue Parking - Étages, places, status en temps réel ■ Responsif - Tous les écrans et orientations ■ Architecture Claire - Séparation models/services/providers/screens

Fichiers de Configuration

pubspec.yaml

- Version: 1.0.0+1 - SDK: ^3.11.0 - Principales dépendances: Firebase, Provider, Google Fonts, Image Picker

analysis_options.yaml

- Options d'analyse de code Dart - Règles de linting

Conclusion

Cette application est une solution complète de gestion de parking utilisant: - Flutter pour le multi-plateforme - Firebase pour le backend - Provider pour la gestion d'état - Architecture moderne avec séparation des responsabilités

Tous les fichiers suivent une structure claire et sont organisés par domaine fonctionnel.

Documentation générée le 29 Mai 2026

Documentation generee le 29 Mai 2026